



DURBAN UNIVERSITY OF TECHNOLOGY
INYUVESI YASETHEKWINI YEZOBUCHWEPHESHE

FACULTY OF ACCOUNTING AND INFORMATICS

DEPARTMENT OF INFORMATION TECHNOLOGY

2024

MID YEAR MAIN EXAMINATION

INSTRUCTIONAL PROGRAMME : B INFORMATION & COMMUNICATION TECHNOLOGY

INSTRUCTIONAL OFFERING : INTEGRATIVE PROGRAMMING AND TECHNOLOGIES III

PAPER NUMBER : 1

SUBJECT CODE/S : IPRT301

DATE : 7 JUNE 2024

DURATION : 3 HOURS

TIME : 14H00 - 17H00

TOTAL MARKS : 100

NUMBER OF PAGES : 9 (INCLUDING COVER PAGE)

EXAMINER/S : BOAMAH-ABU, CHARLES (PhD)

MODERATOR/S : IBIDUN OBAGBUWA, CHRISTIANA (PhD)

INSTRUCTIONS/REQUIREMENTS:-

1. This examination is closed book
2. Answer all questions
3. Where stated "briefly", please answer in a few lines
4. Ensure your handwriting is readable

Do not turn the page until permission is given

Question 1

20 marks

In each of the following write **T** (for **true**) or **F** (for **false**) for your answer.

1. State whether the following is true or false about interfaces.
 - 1.1 Can contain method definitions
 - 1.2 Can contain method declarations
 - 1.3 Can contain constants
 - 1.4 Can contain instance variables
 - 1.5 Mainly used to instantiate objects
 - 1.6 Can implement other interfaces
2. State whether the following is true or false about polymorphism.
 - 2.1 Allows an object to exist in different forms
 - 2.2 Occurs because of inheritance
 - 2.3 Occurs because of composition
 - 2.4 An appropriate behaviour of an object is determined at compile time
 - 2.5 An appropriate behaviour of an object is determined at run time
3. True or false: the following architecture uses a registry mechanism.
 - 3.1 Client-server
 - 3.2 Broker
 - 3.3 Web service
4. True or false: the following is used to define the structure of XML data.
 - 4.1 XML schema
 - 4.2 XSL
 - 4.3 XMLStreamReader
5. True or false: the following architecture uses XML.
 - 5.1 Client-server
 - 5.2 Broker
 - 5.3 Web services

Question 2

10 marks

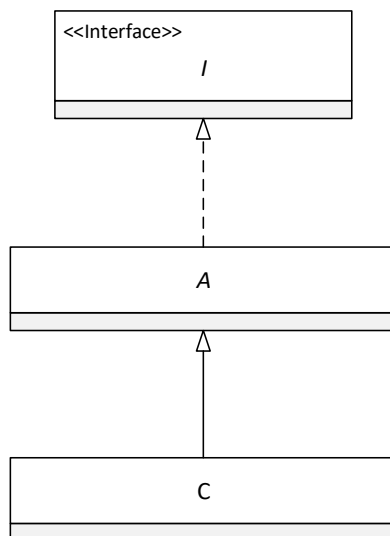
Briefly answer the following.

- 2.1. What is the main challenge with Integrative programming? (2 marks)
- 2.2. Give one reason why XML is important in Integrative programming. (2 marks)
- 2.3. Give one reason why StaX is preferable to DOM. (2 marks)
- 2.4. Give one benefit of design patterns. (2 marks)
- 2.5. What is the main difference between Webservice and Microservice architectures? (2 marks)

Question 3

12 marks

The UML diagram below shows the abstract class **A** implements interface **I** and class **C** inherits **A**.

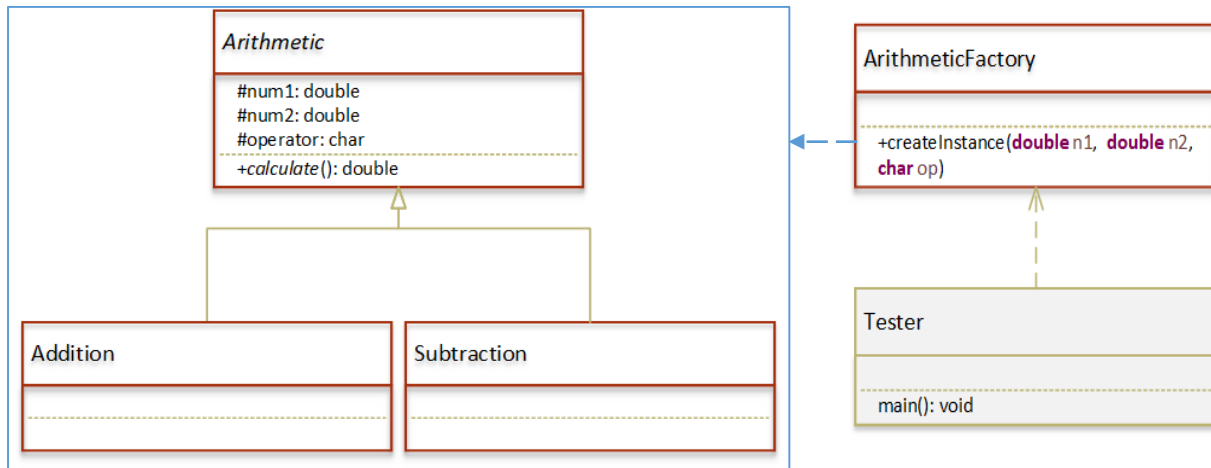


1. In each of the following, state whether the statement is valid or invalid. Explain your answer.
 - 1.1. `I i1=new I();` (2 marks)
 - 1.2. `I i2= new A();` (2 marks)
 - 1.3. `I i3=new C();` (2 marks)
 - 1.4. `A a2=new C();` (2 marks)
2. Using the diagram explain the benefit of code re-use. (2 marks)
3. Using **I** and **A**, explain why “interface is a contract”. (2 marks)

Question 4

18 marks

The UML diagram below shows the classes **Arithmetic**, **Addition**, **Subtraction**, **ArithmeticFactory** and **Tester**.



- Arithmetic** is an abstract class; **num1** and **num2** are numbers; **operator** is a character representing an arithmetic operation, e.g., '+', '-', etc.); and **calculate()** is an abstract method. The class contains appropriate **constructors** and **toString()** method
- Addition** and **Subtraction** inherit **Arithmetic**
 - Addition** overrides **calculate()** to return the sum of **num1** and **num2**. The class contains appropriate **constructors** and **toString()**
 - Subtraction** overrides **calculate()** to return the value when **num2** is subtracted from **num1**. The class contains appropriate **constructors** and **toString()** method
- ArithmeticFactory** is a factory class. **createInstance(double n1, double n2, char op)** takes three parameters: **n1** and **n2** (numbers) and **op** (a character). If the character is
 - '+', the method uses the parameters to create and return an instance of **Addition**.
 - '-', the method uses the parameters to create and return an instance of **Subtraction**.
- Tester** creates an instance of **ArithmeticFactory** and calls its method to return an instance of a subclass (i.e., **Addition** or **Subtraction**) of **ArithmeticFactory**. This subclass is then displayed on the screen.

Use the information above to answer **Questions 1 to 5** below.

- Write the method **calculate()** in the class **Arithmetic**. (2 marks)
- Write the header of the class **Addition**. (2 marks)
- Write the method **calculate()** in the class **Addition**. (3 marks)

4. Complete the method **createInstance**(double n1, double n2, char op) in the class **ArithmeticFactory**. Assume the relevant classes and their methods (including appropriate **constructors**) have been implemented. **(6 marks)**

```
...
...
public class ArithmeticFactory {
    _____createInstance(double n1, double n2, char op){
        _____
        _____
        _____
        _____
    }
}
```

5. Assume all the classes and their methods (including appropriate **constructors** and **toString()** methods) have been implemented. Complete **main()** in **Tester** as follows:

1. Create an instance of **ArithmeticFactory** called **af**. **(1 marks)**
2. Use **af** to call an appropriate method (with the values 30, 20 and '+') to return the instance of a subclass. **(3 marks)**
3. Display the instance on the screen. **(1 mark)**

```
...
...
public class Tester {
    public static void main(String[] args) {
        _____
        _____
        _____
    }
}
```

Question 5

20 marks

library.dtd contains the DTD below, which defines the XML structure in **library.xml**.

```
<!ELEMENT Library (book)*>
<!ELEMENT book (title)*>
<!ATTLIST book isbn CDATA #REQUIRED>
<!ELEMENT title (#PCDATA)>
```

Below is the data in **library.xml**.

isbn	title
001-002-002-1	Java for dummies
001-002-002-2	Design patterns

1. Complete the XML code below for **library.xml**. (5 marks)

```
<?xml version="1.0" encoding="_____"?>
```

2. Write the code in **library.xml** to link to the structure in **library.dtd**. (2 marks)
3. An application to read the XML in **library.xml** consists of the classes **Book**, **ReadBook** and **Tester** (below).

```

Book
  isbn
  title
  Book()
  Book(String, String)
  getIsbn() : String
  getTitle() : String
  setIsbn(String) : void
  setTitle(String) : void
  toString() : String

ReadBook
  ReadBook()
  read(String) : void

Tester
  main(String[]) : void
```

ReadBook reads the XML to create **Book** objects. The method **read(String)** takes a string (a file name) as a parameter to set up an **XMLStreamReader** (called **xmlreader**) to read XML from **library.xml**.

Assume **Book** has been implemented. Below (see **next page**) are parts of the code for the method **read(String file)**. Use this to answer **Questions 3.1 to 3.3**.

- 3.1. Complete the code segment (**marked 3.1**) for a loop to read until end of the XML document. (2 marks)

- 3.2. Complete the code segment (**marked 3.2**) as follows. If the variable **event** is the start element **<book>**, get the value of the **isbn** attribute; create an instance of **Book** and assign to **bk**; and use the value of the attribute to set the **isbn** of **bk**. (7 marks)
- 3.3. Complete the code segment (**marked 3.3**) as follows. If the variable **event** is text, get the text and use it to set the **title** of **bk**. (4 marks)

```
public void read(String file){
    try {
        FileReader fr=new FileReader(file);
        XMLStreamReader xmlreader=null;//
        XMLInputFactory xmlfactory=XMLInputFactory.newInstance();
        xmlreader=xmlfactory.createXMLStreamReader(fr);
        Book bk=null;
        int event;
        //3.1. Complete the code below
        while(_____){
            event = xmlreader.next();
            switch(event) {
                //3.2. Complete the code below
                case_____ : {
                    _____
                    _____
                    _____
                }
                break

                //3.3. Complete the code below
                case_____ : {
                    _____
                    _____
                    _____
                }
                break
                ...
                ...
            }
        }
    }
    ...
}
```

Question 6

20 marks

A client application writes an object of the class **Item** into a stream. A server application reads the object, calculates and writes the total price into the stream. The client application then reads the total price, and displays the object and its total price on the screen. The server **address** is "127.0.0.1" and communication **port** is 1999.

Part 1: Below is the class **Item**. The instance variables are **name** and **price** (of an item).

```
Item
  name
  price
  Item(String, double)
  getName() : String
  getPrice() : double
  setName(String) : void
  setPrice(double) : void
  toString() : String
```

Part 2: Below is the client application. This consists of two classes: **Client** and **ClientTester**

```
Client.java
  Client
    writeAndRead(Item) : double
  ClientTester.java
    ClientTester
      main(String[]) : void
```

Client contains the method **writeAndRead** (Item i). The parameter **i** is the object to write to the stream. This method:

- Sets up the appropriate socket and stream
- Writes **i** to the stream
- Reads a number (total price) from the stream
- Closes the stream and the socket
- Returns the number (total price)
- Handles all possible exceptions

ClientTester contains **main()**, which:

- Creates an instance of **Item** (called **i1**) and **Client** (called **c1**)
- Displays **i1**
- Calls the appropriate method of **c1** to return the total price and display it on the screen.
- Handles all possible exceptions

Below is a typical output.

```
Name=Biscuit, Price=R100.0
Total Price (i.e., Price + VAT (@ 15%)=R115.0
```

Use the information to answer **Questions 1 to 3** below.

1. Complete the class header of **Item** for this distributed application. (2 marks)
2. Complete the method **writeAndRread**(Item i) below to:
 - 2.1. Set up the appropriate socket (2 marks)
 - 2.2. Set up the appropriate stream (4 marks)
 - 2.3. Write **i** to the stream (2 marks)
 - 2.4. Read a number (total price) from the stream (2 marks)
 - 2.5. Close the stream and the socket (1 mark)
 - 2.6. Return the number (total price) (1 mark)
 - 2.7. Handle all possible exceptions (2 marks)

```

public double writeAndRread(Item i) _____{
    _____
    _____
    _____
    _____
    _____
    _____
    _____
}

```

3. Complete the code segment below for **main()** in **ClientTester** which creates an instance of Client (called **c1**) and calls **writeAndRread**(Item i) with an instance of Item (called **i1**). Handle all errors. Assume all the classes and their methods (including appropriate **constructors** and **toString()** methods) have been implemented. (4 marks)

```

...
...
public class ClientTester {
    public static void main(String [] args)_____ {
        Item i1=new Item("Biscuit", 100);
        Client c1=new Client();
        System.out.println(i1);
        System.out.println("Total Price (Price + VAT (@ 15%)=R" + _____);
    }
}

```